produces:

```
60 80 100
```

To avoid this behavior, call freeze on the string literal, which causes the string to be *interned*, meaning the object is allocated once, and future attempts to allocate the same string will result in the same object:

```
3.times do
  print "hello".freeze.object_id, " "
end
```

produces:

```
60 60 60
```

You can achieve this with the frozen_string_literal magic comment. Strings with interpolation are never interned.

String literals are always encoded using the encoding of the source file that contains them, regardless of the content of the string:

```
# encoding: utf-8
def show_encoding(str)
  puts "'#{str}' (size #{str.size}) is #{str.encoding.name}"
end

show_encoding "cat"    # latin 'c', 'a', 't'
show_encoding "ðog"    # greek delta, latin 'o', 'g'
```

produces:

```
'cat' (size 3) is UTF-8
'ðog' (size 3) is UTF-8
```

Symbols and regular expression literals that contain only 7-bit characters are encoded using US-ASCII. Otherwise, they will have the encoding of the file that contains them.

```
# encoding: utf-8
def show_encoding(str)
  puts "#{str.inspect} is #{str.encoding.name}"
end
show_encoding :cat
show_encoding :ðog

show_encoding /cat/
show_encoding /ðog/
```

produces:

```
:cat is US-ASCII
:ðog is UTF-8
/cat/ is US-ASCII
/ðog/ is UTF-8
```

You can create arbitrary Unicode characters in strings and regular expressions using the \u escape. This has two forms: \uxxxx lets you encode a character using four hex digits, and the delimited form \u{x... x... x...} lets you specify a variable number of characters, each with a variable number of hex digits:

```
# encoding: utf-8
"Greek pi: \u03c0"          # => "Greek pi: π"
```